than others that had waited for longer. For instance, you don't always read your emails in the order you received them, but often you skip newsletters or “funny” jokes from friends to go and read work-related messages first. Likewise, in an emergency room, the next case treated is not necessarily going to be one that has been waiting for the longest time. Rather, every case is evaluated at arrival and assigned a priority, and the highest priority one is going to be called in when a doctor becomes available.

That's the idea behind priority queues: they behave like regular, plain queues, only the front of the queue is dynamically determined based on some kind of priority. The differences caused to the implementation by the introduction of priority are profound. Enough to deserve a special kind of data structure.

But that’s not all: we can even define basic containers as *bag* or *stack* as special cases of priority queues: appendix D explores how this is possible. This is an interesting topic to gain a deeper understanding of how priority queues work, although in practice those containers are usually implemented ad-hoc, because we can achieve better performance by leveraging their specific characteristics.

2.5 Concrete Data Structures

Let’s now move from abstract to concrete data structures. Knowing how the API of a priority queue works is good enough to use it, but often it is not enough to use it well.